

Traversability Path

Research Internship

Melinda Tran



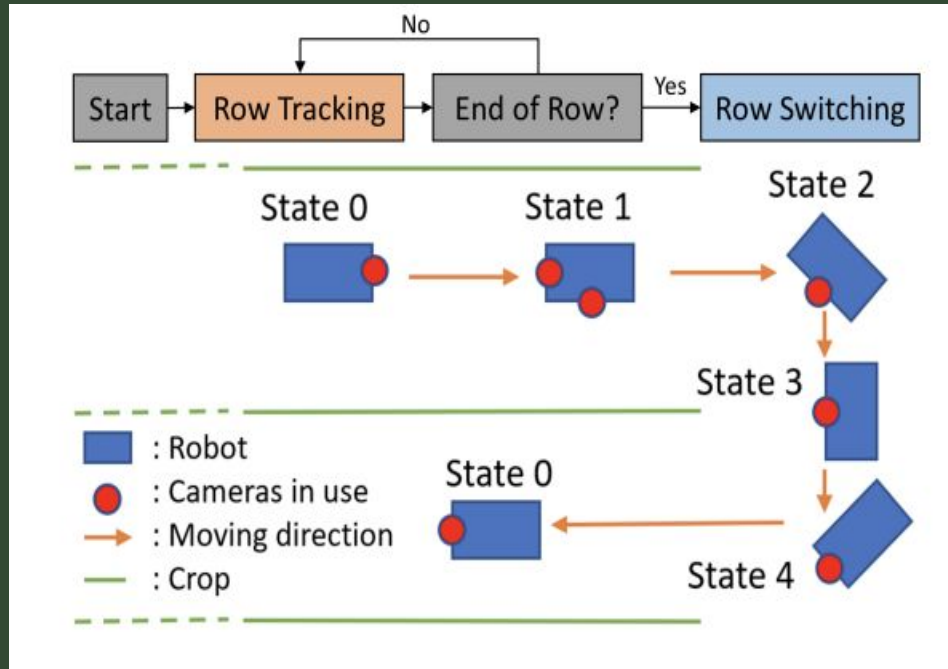
AgAID
Institute

Project Objective:

Program the robot to move
within the traversability path
and achieve localization without
GPS (on simulation)

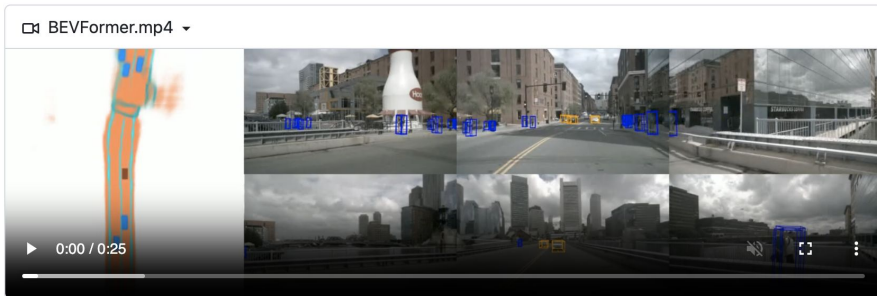


Vision Path Process



- Mainly looking at the row tracking to end of row process
 - State 0 and state 1

BEVFormer: a Cutting-edge Baseline for Camera-based Detection



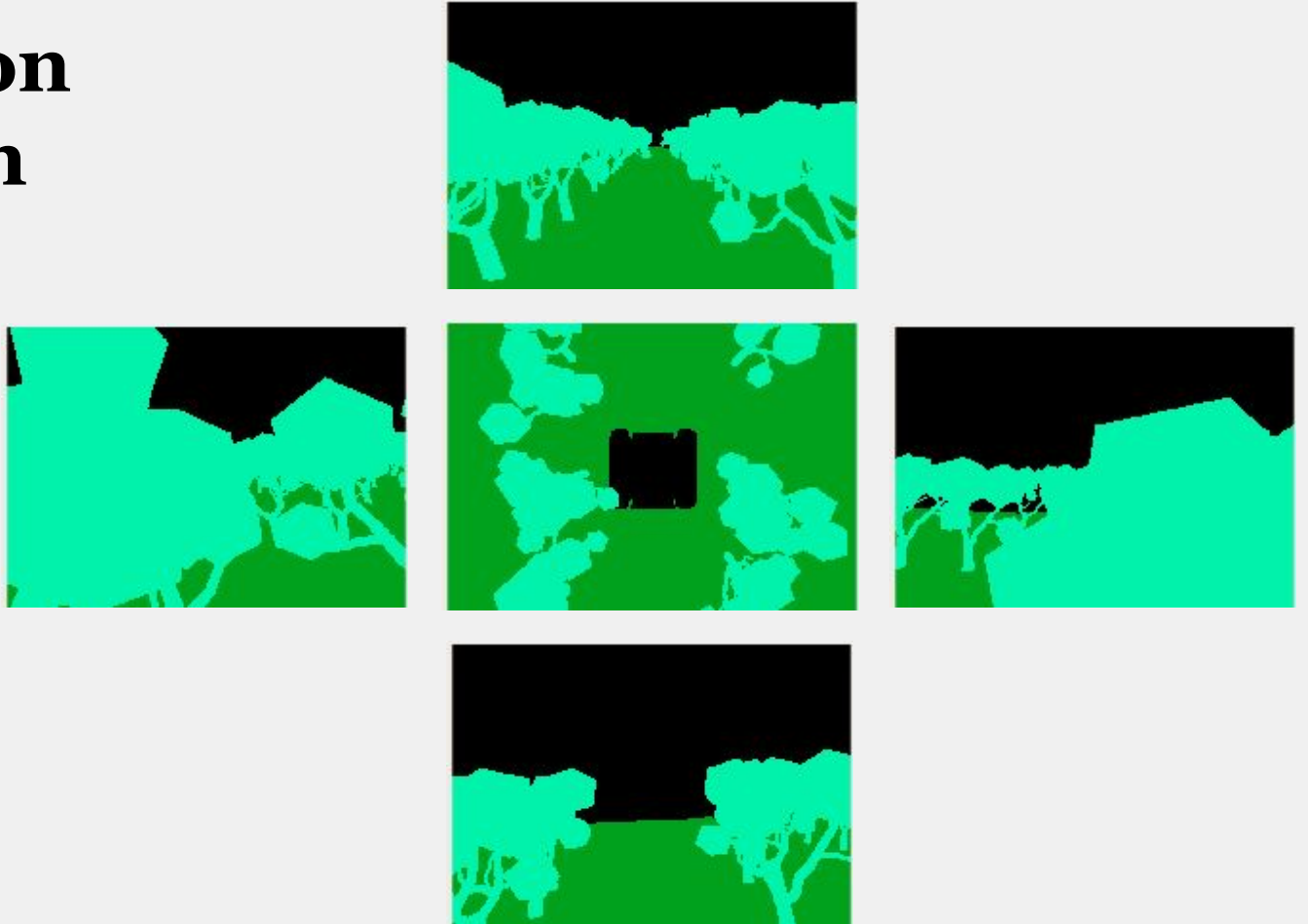
BEVFormer: Learning Bird's-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers, ECCV 2022

- [Paper in arXiv](#) | [Paper in Chinese](#) | [OpenDriveLab](#)
- [Slides in English](#) | [Occupancy and BEV Perception Talk Slides](#)
- [Blog in Chinese](#) | [Video Talk](#) and [Slides](#) (in Chinese)
- [BEV Perception Survey](#) (Accepted by PAMI) | [Github repo](#)

Objective 1 : BEV

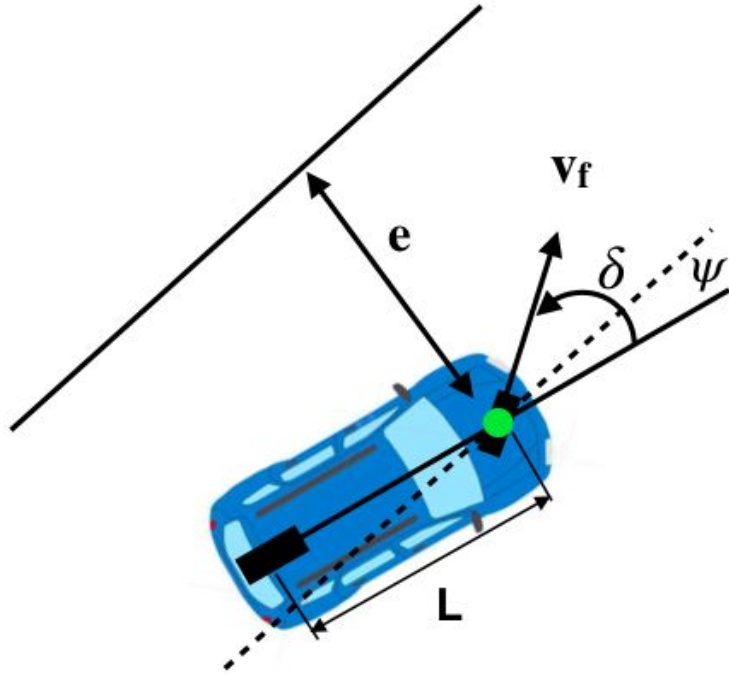
1. Extract surrounding segmented images from Gazebo simulation
2. Fine-tune the BEVFormer model using our custom dataset
3. Implement redirection logic as needed
4. Integrate with the real-world segmentation model

Navigation Approach



Future Work

- Troubleshooting/ finish fine tuning the BEVformer model
- Compute redirection
- Connect with the real world scenario/ model



Objective 2 :

Traversability from front
segmentation camera and
calculations

Segmentation Camera

- Built-in Gazebo sensor, mounted front-facing
- Outputs pixel-level semantic labels: path, trunks, leaves
- Direct input to BEV mapping node



Row Centering

- Row-centering logic:
 - Divide BEV into 25 vertical slices
 - For each slice: detect path pixels → compute midpoint
 - Average across slices = row centerline
- Robot adjusts steering to minimize lateral offset
- Debug overlay shows:
 - Green line = detected centerline
 - Red dots = path sample points
 - EMA (Exponential Moving Average) smoothing to reduce jitter

Stanley Controller

Why ?

- Provides stable heading correction compared to row-centering alone
-

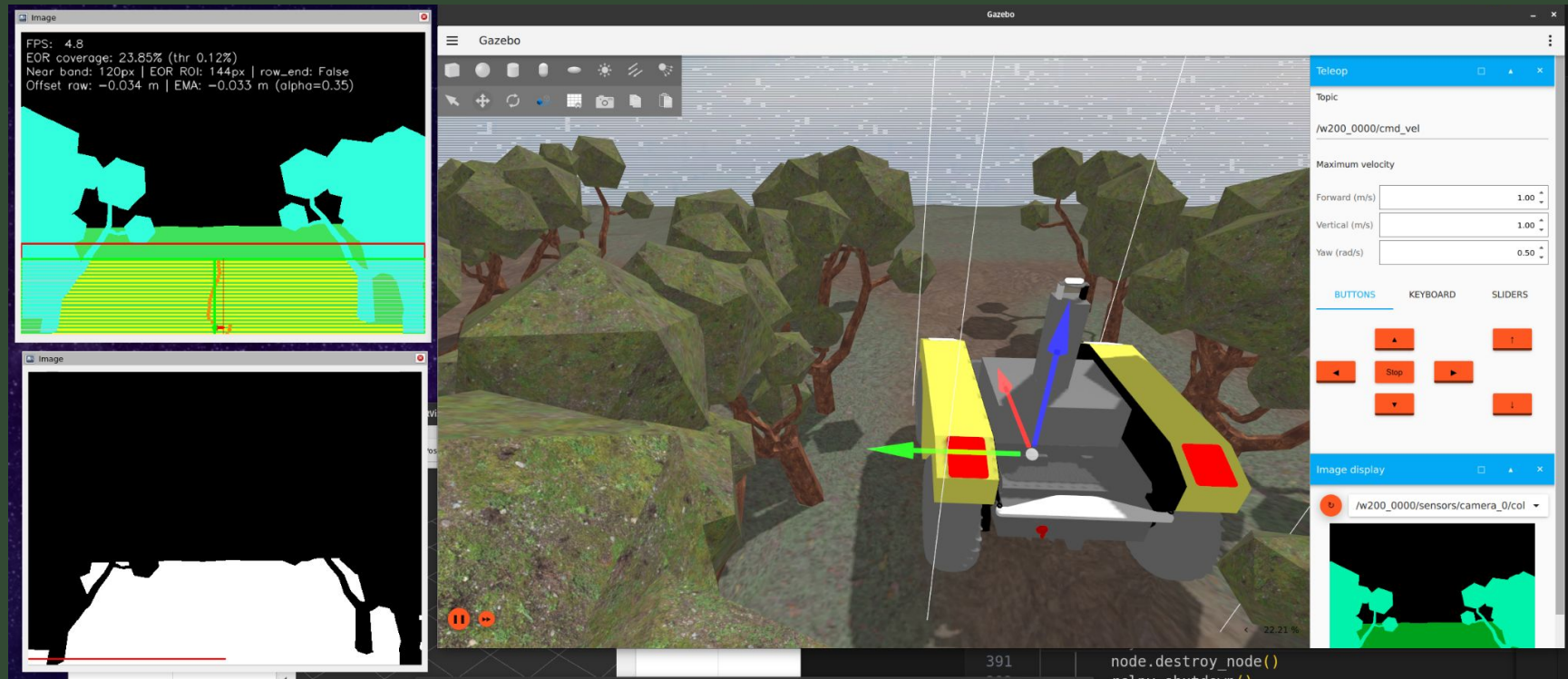
How ?

- Uses control equation:
 - $\delta = \theta_error + \arctan(k * cross_track_error / velocity)$
- Where:
 - θ_error = angular difference to row heading
 - $cross_track_error$ = lateral distance from row centerline
 - k = tuning gain (currently experimenting with α scaling factor)

End of Row Detection

- End-of-row detection logic :
 - Monitors density of path pixels ahead
 - If threshold drops below limit → robot assumes end of row
 - Will trigger row-switching maneuver (future work)

Demonstration



- Alpha (α) controls trade-off between steering responsiveness vs. stability
 - Smaller $\alpha \rightarrow$ slower, smoother corrections
 - Larger $\alpha \rightarrow$ faster, riskier corrections (oscillations)
 - Currently tuning via simulation to find best balance

Refine Control

Accuracy / Metrics

- Cross-track error (CTE):
 - Distance from true row centerline
- Row coverage:
 - % of row successfully navigated before failure
- End-of-row detection accuracy:
 - Correctly identifies row ends
- Debug tools:
 - `/row_metrics/debug_overlay` for visualization

- Implement row turning with Implement row-turning maneuver
 - (reusing prior intern's turning code)
- Extend navigation to multiple connected rows from previous intern
 - integrate full pipeline

Future Work



Thank you !

Mentors: Manoj Karkee & Dawood Ahmed

CALS Lab & AgAID



**AgAID
Institute**